RxRosetta source code

app.py

```
import time, os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from kb import T, SAMPLES
import llm
from engine import analyze, triage, evaluate

app = FastAPI(title="RxRosetta")
EVENTS = []
class Txt(BaseModel): text: str
class Img(BaseModel): data: str; mime: str
class Chk(BaseModel): keys: list[str] = []; symptoms: list[str] = []
class Ev(BaseModel): type: str; detail: str; level: str = "info"

def log(type, detail, level="info"):
    EVENTS.append(dict(t=time.strftime("%H:%M:%S"), type=type, detail=detail, level=level)); del EVENTS[:-60]

@app.post("/api/analyze")
def a(r: Txt): return analyze(r.text)
@app.post("/api/scan")
def scan(r: Img):
    try: txt = llm.scan(r.data, r.mime)
    except RuntimeError: return dict(error="Photo scan needs an API key. Set ANTHROPIC_API_KEY or GEMINI_API_KEY and restart,
    or paste the text instead.")
    except Exception as e: return dict(error=f"Scan failed: {e}")
    return dict(text="" if txt.upper().startswith("NONE") else txt)
@app.post("/api/checkin")
def c(r: Chk):
    lv = triage(r.keys, r.symptoms)
    log("check-in", f"{lv.upper()}: {', '.join(r.symptoms) or 'no symptoms'}", {"ok": "info", "call": "warn", "er":
    "bad"}[lv])
    return dict(level=lv)
@app.post("/api/event")
def e(r: Ev): log(r.type, r.detail, r.level); return {}
@app.get("/api/events")
def ev(): return EVENTS[::-1]
@app.get("/api/i18n")
def i(): return T
@app.get("/api/samples")
def s(): return SAMPLES
@app.get("/api/eval")
def v(): return evaluate()
app.mount("/", StaticFiles(directory=os.path.join(os.path.dirname(__file__), "static"), html=True))
```

engine.py

```
import re, difflib, random
from collections import Counter
from kb import KB, INTER, SLOTS

NAMES = {n: k for k, v in KB.items() for n in [k] + v["al"]}
ORDER = ["morning", "afternoon", "evening", "night"]

def find(l):
    for n, k in sorted(NAMES.items(), key=lambda x: -len(x[0])):
        if re.search(r"\b" + re.escape(n) + r"\b", l): return k, n, "exact"
    for tok in re.findall(r"[a-z]{5,}", l):
        m = difflib.get_close_matches(tok, list(NAMES), 1, 0.8)
        if m: return NAMES[m[0]], tok, "fuzzy"
    return None, None, None

def freq(l, key):
    m = re.search(r"\b([01])\s*-\s*([01])\s*-\s*([01])(?:\s*-\s*([01]))?\b", l)
    if m:
        g = m.groups(); names = ORDER if g[3] else ["morning", "afternoon", "night"]
        return [n for n, v in zip(names, g) if v == "1"]
    for pat, s in [(r"\b(qid|qds|four times|4 times)\b", ORDER),
                   (r"\b(tds|tid|thrice|three times|3 times|8 hourly)\b", ["morning", "afternoon", "night"]),
                   (r"\b(bd|bid|twice|two times|2 times|12 hourly)\b", ["morning", "night"]),
                   (r"\b(hs|bedtime|at night|nocte)\b", ["night"]), (r"\b(sos|prn|as needed)\b", ["asneeded"]),
                   (r"\b(od|once|daily|morning|empty stomach)\b", [(KB.get(key) or {}).get("t", "morning")])]:
        if re.search(pat, l): return s
    return []

def food(l, key):
    for pat, f in [(r"empty stomach", "empty"), (r"before (food|meals?|breakfast)", "before"),
                   (r"after (food|meals?)|\bpc\b", "after"), (r"with (food|meals?)", "with")]:
        if re.search(pat, l): return f
    return (KB.get(key) or {}).get("food", "any")

def interactions(keys):
    ks = set(keys); out = []
    for a, b, sev, eff in INTER:
```

```
        if a in ks and b in ks: out.append(dict(a=a, b=b, sev=sev, eff=eff))
    for k, c in Counter(keys).items():
        if c > 1: out.append(dict(a=k, b=k, sev="mod", eff="dupe"))
    if {"pantoprazole", "omeprazole"} <= ks: out.append(dict(a="pantoprazole", b="omeprazole", sev="low", eff="ppi"))
    return out

def analyze(text):
    meds = []
    for raw in re.split(r"[\n;]+", text):
        l = raw.strip().lower()
        if not l: continue
        key, typed, how = find(l)
        if not key:
            m = re.match(r"(?:tab|tabs|tablet|cap|capsule|syp|inj)\.?\s+([a-z]+)", l)
            if not m: continue
            typed = m.group(1)
        d = re.search(r"(\d+(?:\.\d+)?)\s*(mg|mcg|g|ml)\b", l)
        slots = freq(l, key); v = KB.get(key, {})
        meds.append(dict(key=key, name=(key or typed).title(), typed=typed, known=bool(key), how=how,
            dose=f"{d.group(1)} {d.group(2)}" if d else None, slots=slots, food=food(l, key), cat=v.get("cat"),
            notes=v.get("n", []), verify=(not key) or (not d) or (not slots) or how == "fuzzy",
            link=f"https://dailymed.nlm.nih.gov/dailymed/search.cfm?labeltype=all&query={key or typed}"))
    return dict(meds=meds, issues=interactions([m["key"] for m in meds if m["key"]]))

THIN = {"warfarin", "aspirin", "clopidogrel", "ibuprofen"}
def triage(keys, sym):
    s, k = set(sym), set(keys)
    if s & {"chest", "breath", "swell", "faint"}: return "er"
    if "bleed" in s and k & THIN: return "er"
    if "low" in s and "glimepiride" in k: return "er"
    return "call" if s else "ok"

def evaluate(n=40, seed=7):
    R = random.Random(seed); ALL = ORDER
    F = [("OD", None), ("once daily", None), ("BD", ["morning", "night"]), ("twice daily", ["morning", "night"]),
         ("TDS", ["morning", "afternoon", "night"]), ("1-0-1", ["morning", "night"]), ("0-0-1", ["night"]),
         ("HS", ["night"]), ("QID", ALL)]
    tp = fp = fn = dose = slot = hit = 0; ir = [0, 0]; fails = []
    for i in range(n):
        gold, lines = {}, []
        for k in R.sample(list(KB), R.randint(4, 7)):
            v = KB[k]; d = R.choice(v["d"]); u = v.get("u", "mg"); name = R.choice([k] + v["al"])
            if R.random() < .25 and len(name) > 5:
                j = R.randrange(1, len(name) - 2); name = name[:j] + name[j + 1] + name[j] + name[j + 2:]
            if R.random() < .15: name = name.replace("i", "l", 1)   # OCR-style confusion
            if R.random() < .10: name = name.replace("o", "0", 1)   # OCR-style confusion
            f, s = R.choice(F); s = s or [v.get("t", "morning")]
            fmt = R.choice(["Tab {n} {d} {u} {f}", "T. {n} {d}{u} - {f}", "{n} {d} {u} ({f})", "Cap {n} {d} {u} {f} x {x}
    days"])
            lines.append(fmt.format(n=name.title() if R.random() < .5 else name.upper(), d=d, u=u, f=f, x=R.randint(3, 30)))
            gold[k] = (f"{d} {u}", s)
        got = {m["key"]: m for m in analyze("\n".join(lines))["meds"] if m["key"]}
        for k, (gd, gs) in gold.items():
            if k in got:
                tp += 1; hit += 1; dose += got[k]["dose"] == gd; slot += got[k]["slots"] == gs
                if got[k]["dose"] != gd or got[k]["slots"] != gs: fails.append(f"case {i}: {k} parsed as {got[k]['dose']}
    {got[k]['slots']}, expected {gd} {gs}")
            else:
                fn += 1; fails.append(f"case {i}: missed {k}")
        fp += len(set(got) - set(gold))
        gp = {(x["a"], x["b"], x["eff"]) for x in interactions(list(gold))}
        pp = {(x["a"], x["b"], x["eff"]) for x in interactions(list(got))}
        ir[0] += len(gp & pp); ir[1] += len(gp)
    r = lambda a, b: round(100 * a / b, 1) if b else 100.0
    return dict(cases=n, medicines=tp + fn, drug_recall=r(tp, tp + fn), drug_precision=r(tp, tp + fp), dose_accuracy=r(dose,
    hit),
        schedule_accuracy=r(slot, hit), interaction_recall=r(*ir), failures=fails[:8],
        note="Synthetic notes (seeded) with brand names, typos and mixed formats. This measures the rule-based parser
    baseline, not real hospital documents.")
```

llm.py

```
"""Optional vision step: an LLM only TRANSCRIBES medicine lines. All checking is done by the deterministic engine."""
import os, httpx
PROMPT = ("Transcribe ONLY the medicine lines from this discharge summary or prescription, one per line, as written, in the
    form "
          "'Tab <name> <dose> <frequency such as 1-0-1 or BD> <food instruction> x <days>'. Keep spellings as written. "
          "Do not add, correct, explain or infer anything. If nothing is legible, reply NONE.")

def scan(b64, mime):
    ak, gk = os.getenv("ANTHROPIC_API_KEY"), os.getenv("GEMINI_API_KEY")
    if ak:
        kind = "document" if mime == "application/pdf" else "image"
        r = httpx.post("https://api.anthropic.com/v1/messages", timeout=90,
            headers={"x-api-key": ak, "anthropic-version": "2023-06-01"},
            json={"model": os.getenv("ANTHROPIC_MODEL", "claude-sonnet-5-5"), "max_tokens": 1000, "messages": [{"role":
    "user", "content": [
                {"type": kind, "source": {"type": "base64", "media_type": mime, "data": b64}}, {"type": "text", "text":
```

```
    PROMPT}]}]})
        r.raise_for_status(); return "".join(c.get("text", "") for c in r.json()["content"]).strip()
    if gk:
        m = os.getenv("GEMINI_MODEL", "gemini-2.0-flash")
        r = httpx.post(f"https://generativelanguage.googleapis.com/v1beta/models/{m}:generateContent?key={gk}", timeout=90,
            json={"contents": [{"parts": [{"text": PROMPT}, {"inline_data": {"mime_type": mime, "data": b64}}]}]})
        r.raise_for_status(); return r.json()["candidates"][0]["content"]["parts"][0]["text"].strip()
    raise RuntimeError("no_key")
```

kb.py

```
"""Curated knowledge base. Every warning shown in the app comes from here (never invented by an LLM).
Verify against official labels (DailyMed links are shown in the UI) before any real-world use."""
KB = {
 "metformin":    dict(al=["glycomet","glucophage"], cat="sugar", food="with", n=["alcohol"], d=[500,850,1000]),
 "glimepiride":  dict(al=["amaryl"], cat="sugar", food="with", n=[], d=[1,2,3]),
 "amlodipine":   dict(al=["amlong","norvasc"], cat="bp", food="any", n=[], d=[2.5,5,10]),
 "lisinopril":   dict(al=["listril","zestril"], cat="bp", food="any", n=["ksalt"], d=[2.5,5,10,20]),
 "metoprolol":   dict(al=["metolar","betaloc"], cat="bp", food="with", n=[], d=[25,50,100]),
 "atorvastatin": dict(al=["atorva","lipitor"], cat="chol", food="any", n=["grape"], t="night", d=[10,20,40]),
 "aspirin":      dict(al=["ecosprin","disprin"], cat="thin", food="after", n=[], d=[75,150]),
 "clopidogrel":  dict(al=["clopilet","plavix"], cat="thin", food="any", n=[], d=[75]),
 "warfarin":     dict(al=["warf","coumadin"], cat="thin", food="any", n=["vitk"], t="night", d=[1,2,3,5]),
 "ibuprofen":    dict(al=["brufen","advil"], cat="pain", food="with", n=[], d=[200,400]),
 "paracetamol":  dict(al=["dolo","crocin","acetaminophen"], cat="pf", food="any", n=["alcohol"], d=[500,650]),
 "pantoprazole": dict(al=["pan","pantocid"], cat="acid", food="before", n=[], d=[20,40]),
 "omeprazole":   dict(al=["omez","prilosec"], cat="acid", food="before", n=[], d=[20,40]),
 "furosemide":   dict(al=["lasix"], cat="water", food="any", n=[], d=[20,40]),
 "spironolactone":dict(al=["aldactone"], cat="water", food="any", n=["ksalt"], d=[25,50]),
 "levothyroxine":dict(al=["thyronorm","eltroxin"], cat="thyroid", food="empty", n=["dairy"], u="mcg", d=[25,50,75,100]),
 "amoxicillin":  dict(al=["mox","amoxil"], cat="abx", food="any", n=[], d=[250,500]),
 "ciprofloxacin":dict(al=["ciplox","cipro"], cat="abx", food="any", n=["dairy"], d=[250,500]),
}
# (drug a, drug b, severity, effect code)
INTER = [
 ("warfarin","aspirin","high","bleed"),("warfarin","ibuprofen","high","bleed"),("warfarin","clopidogrel","high","bleed"),
 ("warfarin","ciprofloxacin","high","inr"),("warfarin","amoxicillin","mod","inr"),
 ("aspirin","ibuprofen","mod","bleed"),("clopidogrel","ibuprofen","mod","bleed"),("clopidogrel","aspirin","low","dual"),
 ("clopidogrel","omeprazole","mod","reduced"),("lisinopril","spironolactone","high","potassium"),
 ("lisinopril","ibuprofen","mod","kidney"),("furosemide","ibuprofen","mod","kidney"),
 ("ciprofloxacin","glimepiride","mod","lowsugar"),("metoprolol","glimepiride","mod","mask"),
]
SLOTS = ["morning","afternoon","evening","night","asneeded"]
L = ["en","ta","hi"]
_R = {
 "slot": {"morning":("Morning","????","????"),"afternoon":("Afternoon","??????","?????"),"evening":("Evening","????","???"),
          "night":("Night","????","???"),"asneeded":("If needed","?????????????","?????? ????? ??")},
 "food": {"before":("Before food","???????? ????","???? ?? ????"),"after":("After food","?????????? ?????","???? ?? ???"),
          "with":("With food","???????","???? ?? ???"),"empty":("Empty stomach","?????? ?????????","???? ???"),
          "any":("Anytime","??????? ??????????????","??? ??")},
 "cat": {"sugar":("Controls blood sugar","??????? ??????????? ???????????????","???? ???? ????????? ???? ??"),
         "bp":("Lowers blood pressure","????? ???????????? ??????????","???? ?????? ?? ???? ??"),
         "chol":("Lowers cholesterol","????????????? ?????????","???????????? ????? ??"),
         "thin":("Prevents blood clots","??????? ???????? ?????????","??? ???? ?? ????? ??"),
         "pain":("Eases pain and swelling","???, ??????????? ??????????","???? ?? ???? ?? ???? ??"),
         "pf":("Eases pain and fever","???, ??????????? ??????????","???? ?? ????? ?? ???? ??"),
         "acid":("Reduces stomach acid","???????????????? ??????????","??????? ?? ???? ??"),
         "water":("Removes extra body water","?????? ???????? ???? ???????????","???? ?? extra ???? ??????? ??"),
         "thyroid":("Replaces thyroid hormone","????????? ???????? ???????????","??????? ??????? ?? ??? ???? ???? ??"),
         "abx":("Fights bacterial infection","?????????? ?????? ???????????","?????????? ??????? ?? ????? ??")},
 "note": {"alcohol":("Avoid alcohol","??????? ???????????","???? ?? ????"),
          "grape":("Avoid large amounts of grapefruit juice","???? ????????????? ?????? ????????????","?????? ?????????? ??? ?? ????"),
          "vitk":("Keep green leafy vegetables steady each week","????? ????? ?????????? ??????????? ??? ?????? ???????????","??? ???????? ???????? ?? ????? ?? ???? ?????? ??? ????"),
          "ksalt":("Avoid potassium salt substitutes","???????????? ????? ???????????? ????????????","???????? ???? ??? ?????? ? ???"),
          "dairy":("Keep 2 hours apart from milk or calcium","????, ????????????????? 2 ??? ????? ??????? ???????","??? ?? ???????? ?? 2 ???? ?? ???? ????")},
 "eff": {"bleed":("Higher risk of bleeding","????????????? ?????? ??????","??? ???? ?? ???? ??????"),
         "inr":("Blood-thinning effect may rise","???????? ?????????? ?????? ???????","??? ???? ???? ?? ??? ??? ???? ??"),
         "dual":("Often prescribed together on purpose; watch for bleeding","???????????? ???????? ?????????; ??????????????? ???????????","????? ???-????? ??? ?? ???? ???; ??? ???? ?? ????? ???"),
         "reduced":("One medicine may work less well","??? ????????? ?????????? ?????????","?? ??? ?? ??? ?? ???? ??"),
         "potassium":("Potassium in blood can get too high","???????????? ???????????? ???????????","??? ??? ???????? ???? ??? ???? ??"),
         "kidney":("Can strain the kidneys","????????????????????? ???? ??????","????? ?? ???? ??? ???? ??"),
         "lowsugar":("Risk of low blood sugar","???????? ??????? ???????? ??????","???? ???? ???? ?? ???? ?? ????"),
         "mask":("May hide warning signs of low sugar","?????????? ???????? ?????????? ??????????","???? ?? ???? ?? ????? ??? ???? ???"),
         "dupe":("Same medicine listed twice","??? ??????? ??? ???? ??????","?? ?? ??? ?? ??? ???? ??"),
         "ppi":("Two acid reducers listed; usually one is enough","?????? ???? ?????????? ?????; ??????? ????? ??????","?? ??????? ????? ???; ????? ?? ?? ???? ??")},
 "sym": {"chest":("Chest pain or pressure","?????? ??? / ????????","???? ??? ???? / ????"),
         "breath":("Trouble breathing","????????? ??????","???? ???? ??? ?????"),
         "swell":("Swelling of face or lips","????? / ???? ???????","????? / ???? ?? ????"),
         "faint":("Fainting","???????","??????"),
         "bleed":("Unusual bleeding or black stools","??????? ????????????? / ??????? ????","???????? ??? ???? / ???? ??"),
```

```
        "low":("Shaky, sweaty or confused","????????, ???????, ????????","??????, ????? ?? ????"),
        "vomit":("Repeated vomiting","????? ??????","???-??? ?????"),
        "fever":("Fever","?????????","?????"),
        "missed":("Missed two or more doses","?????? ?????? ?????? ???? ??????? ?????","?? ?? ?????? ????? ?????")},
 "lvl": {"ok":("Looks stable. Keep following your plan.","???? ??????? ??????. ??????????? ???????????.","?????? ????? ??? ????? ???? ?????"),
        "call":("Call your doctor today.","????? ?????????? ??????????.","?? ?? ?????? ?? ??? ?????"),
        "er":("Get emergency care now.","???? ???? ???????? ???????.","????? ????????? ??? ????")},
 "ui": {"tag":("Your discharge papers, made clear and safe","?????? ?????????? ????????, ??????????? ???????????????","???? ????????? ?? ?????, ???? ?? ????????"),
      "t_in":("Papers","????????","???????"),"t_plan":("My plan","??? ???????","???? ?????"),
      "t_tb":("Teach-back","??????? ?????","??? ?? ????"),"t_ck":("Daily check-in","?????? ????","????? ????"),
      "t_cg":("Caregiver","?????????????","???????????"),
      "go":("Make my plan","??? ????????? ?????????","???? ????? ????? ?????"),
      "speak":("Read aloud","????????? ??????","????? ??????"),
      "safety":("Safety checks","?????????? ????????","??????? ????"),
      "verify":("Verify with your pharmacist","????????????? ??????????????","???????????? ?? ?????? ????"),
      "none":("No known issues found in our list","?????? ?????????? ????????? ?????","????? ???? ??? ??? ?????? ????"),
      "disc":("Decision support only. This is not medical advice.","??? ???????????? ?????? ????? ???????; ???????? ?????? ????.","?? ???? ????? ????? ??, ???????? ???? ?????"),
      "check":("Check my symptoms","???????????? ????","????? ??????"),
      "tick":("Tick anything you feel today","????? ?????? ???????????? ????????????????","?? ?? ????? ?? ??? ?????"),
      "score":("Score","?????????","???"),"ok":("Correct","???","???"),"no":("Not quite","?????????","??? ????"),
      "q_time":("When do you take {d}?","{d} ??????? ??????? ?????? ?????????","{d} ?? ???? ???"),
      "q_why":("What is {d} for?","{d} ????????","{d} ?????? ???"),
      "q_food":("How should you take {d}?","{d} ??????? ??????? ?????? ?????????","{d} ???? ???? ???"),
      "alert":("Your caregiver has been told to help you review","?????? ???????????????????? ????????????????","???? ???????????? ?? ??? ???? ??? ??"),
      "taken":("Taken","?????????","?? ??"),"retry":("Try again","???????? ???????","??? ????? ????"),
      "s_high":("High risk","???? ??????","?????? ????"),"s_mod":("Be careful","????? ????","??????? ????"),"s_low":("Good to know","??????? ?????","????? ?????"),
      "n_med":("Medicines","??????????","???????"),"n_alert":("Safety alerts","?????????????","??????????"),"n_ver":("To verify","??????????","?????? ?? ???"),
      "empty":("Add your papers first","??????? ?????????? ???????????","???? ??????? ??????"),
      "live":("Live updates","????? ????????","???? ?????")},
}
T = {l: {g: {k: v[i] for k, v in d.items()} for g, d in _R.items()} for i, l in enumerate(L)}
SAMPLES = {
 "Heart attack discharge": "Tab Ecosprin 75 mg 0-1-0 after food\nTab Clopilet 75 mg 1-0-0\nTab Atorva 40 mg 0-0-1\nTab Metolar 25 mg 1-0-1\nTab Pan 40 mg 1-0-0 before food\nTab Omez 20 mg OD\nTab Brufen 400 mg TDS after food x 5 days",
 "Diabetes and blood pressure": "Tab Glycomet 500 mg 1-0-1 after food\nTab Amaryl 2 mg 1-0-0\nTab Amlong 5 mg 1-0-0\nTab Lisinopril 5 mg OD\nTab Spironolactone 25 mg OD\nTab Thyronorm 50 mcg empty stomach\nTab Dolo 650 mg SOS",
 "Blood thinner and antibiotic": "Tab Warfarin 3 mg 0-0-1\nTab Ciplox 500 mg BD x 5 days\nTab Paracetamol 650 mg TDS\nTab Lasix 40 mg 1-0-0\nCap Amoxicillin 500 mg TDS x 7 days",
}
```

static/index.html

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>RxRosetta</title>
<link href="https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,600&family=DM+Sans:wght@400;500;600&display=swap" rel="stylesheet">
<style>
:root{--bg:#eef2f0;--card:#fbfcfb;--ink:#27373a;--mut:#6d7f82;--line:#dde5e2;--sage:#7f9d8c;--sage2:#4f7263;--mist:#dce8e2;--blue:#5f7f9a;--sky:#e1eaf1;--amber:#8f6f30;--amberbg:#f4ecdb;--rose:#9a5159;--rosebg:#f3e3e4;--r:18px;--sh:0 1px 2px rgba(39,55,58,.05),0 8px 24px -12px rgba(39,55,58,.12)}
*{box-sizing:border-box}body{margin:0;color:var(--ink);font:16px/1.55 "DM Sans",system-ui,sans-serif;min-height:100vh;
background:radial-gradient(900px 420px at 8% -6%,#d6e6dd 0,transparent 62%),radial-gradient(800px 420px at 98% 0,#d9e6f0 0,transparent 58%),var(--bg) fixed}
h1,h2,h3{font-family:Fraunces,Georgia,serif;font-weight:600;margin:0;letter-spacing:-.01em}h2{font-size:22px}
svg{width:1em;height:1em;stroke:currentColor;fill:none;stroke-width:1.8;stroke-linecap:round;stroke-linejoin:round;flex:none}
.wrap{max-width:1080px;margin:auto;padding:0 24px}
header{padding:26px 0 6px;display:flex;flex-wrap:wrap;gap:14px;align-items:center;justify-content:space-between}
.brand{display:flex;gap:14px;align-items:center}.logo{width:48px;height:48px;border-radius:15px;background:linear-gradient(135deg,#7f9d8c,#5f7f9a);color:#fff;display:grid;place-items:center;font-size:25px;box-shadow:var(--sh)}
h1{font-size:30px;color:var(--sage2);line-height:1.1}.brand p{margin:2px 0 0;color:var(--mut);font-size:14.5px}
.seg{background:var(--card);border:1px solid var(--line);border-radius:999px;padding:4px;display:flex;gap:2px;box-shadow:var(--sh)}
.seg button{font:inherit;border:0;background:none;color:var(--ink);padding:7px 15px;border-radius:999px;cursor:pointer}.seg button.on{background:var(--sage2);color:#fff}
nav{display:flex;gap:8px;flex-wrap:wrap;margin:16px 0 22px;align-items:center}
nav button{font:inherit;display:flex;gap:9px;align-items:center;border:1px solid var(--line);background:var(--card);color:var(--ink);padding:6px 16px 6px 7px;border-radius:999px;cursor:pointer;transition:.15s}
nav button i{font-style:normal;width:26px;height:26px;border-radius:50%;background:var(--mist);color:var(--sage2);display:grid;place-items:center;font-size:13px;font-weight:600}
nav button.on{background:var(--sage2);border-color:var(--sage2);color:#fff}nav button.on i{background:rgba(255,255,255,.22);color:#fff}
nav button.ghost{margin-left:auto;padding:6px 14px;color:var(--mut);background:transparent}nav button.ghost.on{color:#fff;background:var(--sage2)}
button:focus-visible,textarea:focus-visible,label:focus-within{outline:3px solid var(--blue);outline-offset:2px}
.card{background:var(--card);border:1px solid var(--line);border-radius:var(--r);padding:22px;box-shadow:var(--sh)}
.two{display:grid;grid-template-columns:1.55fr 1fr;gap:18px;align-items:start}@media(max-width:860px){.two{grid-template-columns:1fr}}
.lbl{font-size:13px;color:var(--mut);margin:14px 0 8px}
.btn{font:inherit;display:inline-flex;gap:8px;align-items:center;border:1px solid var(--line);background:#fff;color:var(--ink);padding:10px 18px;border-
```

```
    radius:999px;cursor:pointer;transition:.15s}.btn:hover{border-color:var(--sage)}
.btn.primary{background:var(--sage2);color:#fff;border-color:var(--sage2);font-weight:600;padding:11px
    24px}.btn.primary:hover{background:#456757}
.chips{display:flex;gap:8px;flex-wrap:wrap}.try{font:inherit;font-size:14px;border:1px dashed
    var(--sage);background:transparent;color:var(--sage2);padding:6px 14px;border-
    radius:999px;cursor:pointer}.try:hover{background:var(--mist)}
textarea{width:100%;min-height:200px;border:1px solid var(--line);border-radius:14px;padding:16px;font:14.5px/1.7 ui-
    monospace,Menlo,Consolas,monospace;background:#fff;color:var(--ink);resize:vertical}
.row{display:flex;gap:10px;flex-wrap:wrap;margin-top:14px;align-items:center}
.step{display:flex;gap:12px;padding:10px 0}.step b{display:block}.step span{color:var(--mut);font-size:14.5px}
.ic{width:38px;height:38px;border-radius:12px;display:grid;place-items:center;font-size:19px;background:var(--
    mist);color:var(--sage2);flex:none}.ic.b{background:var(--sky);color:var(--blue)}.ic.a{background:var(--
    amberbg);color:var(--amber)}
.safe div{display:flex;gap:10px;align-items:center;padding:6px 0;font-size:14.5px}.safe svg{color:var(--sage2);font-
    size:18px}
.stats{display:grid;grid-template-columns:repeat(3,1fr);gap:14px;margin:16px 0}@media(max-width:620px){.stats{grid-template-
    columns:1fr}}
.stat{display:flex;gap:14px;align-items:center;padding:16px 18px}.stat .n{font:600 30px/1
    Fraunces,Georgia,serif;color:var(--sage2)}.stat small{color:var(--mut);display:block;font-size:14px}
.top{display:flex;justify-content:space-between;align-items:center;flex-wrap:wrap;gap:10px}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(250px,1fr));gap:16px}
.slot{padding:0;overflow:hidden}.slot .hd{display:flex;gap:12px;align-items:center;padding:16px
    18px;background:var(--t,var(--mist))}.slot .hd h3{font-size:19px;flex:1}.slot .hd .ic{background:rgba(255,255,255,.65)}
.slot .bd{padding:6px 18px 12px}.med{padding:14px 0;border-top:1px solid var(--line)}.med:first-child{border-top:0}
.mh{display:flex;gap:12px;align-items:center}.av{width:38px;height:38px;border-radius:50%;display:grid;place-
    items:center;font:600 16px Fraunces,Georgia,serif;flex:none}.mh>div{flex:1;min-width:0}.mh b{font-
    size:17px;display:block;line-height:1.25}.mh small{color:var(--mut)}
.take{font-size:13px;border:1px solid var(--line);padding:4px 11px;border-
    radius:99px;cursor:pointer;color:var(--mut);background:#fff;display:inline-flex;gap:6px;align-items:center}.take
    input{accent-color:var(--sage2)}.take:has(input:checked){background:var(--mist);color:var(--sage2);border-
    color:var(--sage)}
.chip{display:inline-block;font-size:13px;padding:2px 10px;border-
    radius:99px;background:var(--mist);color:var(--sage2);margin:8px 6px 0
    0}.chip.v{background:var(--amberbg);color:var(--amber)}.chip.n{background:var(--sky);color:#476683}
.iss{display:flex;gap:14px;padding:14px 16px;margin-top:12px;border-radius:14px;background:var(--mist);align-items:flex-
    start}.iss.high{background:var(--rosebg)}.iss.mod{background:var(--amberbg)}
.iss .ic{background:rgba(255,255,255,.7)}.iss.high .ic{color:var(--rose)}.iss.mod .ic{color:var(--amber)}.iss b{text-
    transform:capitalize}.iss a{color:var(--blue);font-size:14px}
.badge{font-size:12px;font-weight:600;padding:2px 9px;border-radius:99px;margin-
    left:8px;background:rgba(255,255,255,.75)}.iss.high .badge{color:var(--rose)}.iss.mod .badge{color:var(--amber)}.iss
    .badge{color:var(--sage2)}
.mut{color:var(--mut)}.disc{font-size:13px;color:var(--mut);margin:22px 0 40px;text-align:center}
.seq{display:flex;gap:6px;margin-bottom:18px}.seq i{flex:1;height:6px;border-radius:9px;background:var(--line)}.seq
    i.d{background:var(--sage)}.seq i.c{background:var(--blue)}
.q{max-width:680px;margin:auto}.q h2{font-size:26px;margin-bottom:14px}
.opt{display:flex;gap:14px;align-items:center;width:100%;text-align:left;margin:10px 0;padding:14px 16px;border-
    radius:14px;border:1px solid var(--line);background:#fff;font:inherit;cursor:pointer;transition:.15s}.opt:hover{border-
    color:var(--sage)}
.opt i{font-style:normal;width:30px;height:30px;border-
    radius:50%;background:var(--mist);color:var(--sage2);display:grid;place-items:center;font-weight:600;font-
    size:14px;flex:none}
.opt.good{background:var(--mist);border-color:var(--sage2)}.opt.bad{background:var(--rosebg);border-color:var(--rose)}
.ring{width:150px;height:150px;border-radius:50%;background:conic-gradient(var(--sage2) calc(var(--p)*1%),var(--line)
    0);display:grid;place-items:center;margin:8px auto 16px}.ring span{width:118px;height:118px;border-
    radius:50%;background:var(--card);display:grid;place-items:center;font:600 34px
    Fraunces,Georgia,serif;color:var(--sage2)}
.syms{display:grid;grid-template-columns:repeat(auto-fit,minmax(250px,1fr));gap:12px;margin-top:14px}
.sym{display:flex;gap:12px;align-items:center;padding:14px 16px;border:1px solid var(--line);border-
    radius:14px;background:#fff;cursor:pointer}.sym input{width:20px;height:20px;accent-
    color:var(--sage2)}.sym:has(input:checked){background:var(--amberbg);border-color:#d9c08a}
.banner{display:flex;gap:12px;align-items:center;padding:16px 18px;border-radius:14px;font-weight:600;margin-top:16px}.banner
    .ok{background:var(--mist);color:var(--sage2)}.banner.call{background:var(--amberbg);color:var(--
    amber)}.banner.er{background:var(--rosebg);color:var(--rose)}.banner svg{font-size:22px}
.ev{display:flex;gap:14px;padding:12px 0;border-top:1px solid var(--line);align-items:flex-start}.ev:first-child{border-
    top:0}.dot{width:12px;height:12px;border-radius:50%;background:var(--sage);margin-top:7px;flex:none}.ev.warn
    .dot{background:#c3a05a}.ev.bad .dot{background:var(--rose)}
.bar{height:8px;border-radius:9px;background:var(--line);margin-top:10px;overflow:hidden}.bar
    i{display:block;height:100%;background:linear-gradient(90deg,var(--sage),var(--sage2));border-radius:9px}
.big{font:600 34px/1 Fraunces,Georgia,serif;color:var(--sage2)}.empty{text-align:center;padding:46px 20px}.empty .ic{margin:0
    auto 14px;width:56px;height:56px;font-size:26px}
@media (prefers-reduced-motion:no-preference){main>*{animation:in .35s ease-out both}@keyframes
    in{from{opacity:0;transform:translateY(6px)}}}
</style></head><body><div class="wrap">
<header><div class="brand"><div class="logo" id="logo"></div><div><h1>RxRosetta</h1><p id="tag"></p></div></div><div
    class="seg" id="lang"></div></header>
<nav id="nav"></nav><main id="main"></main></div>
<script>
const $=s=>document.querySelector(s),LN={en:"English",ta:"?????",hi:"??????"},VC={en:"en-IN",ta:"ta-IN",hi:"hi-IN"};
const P=d=>`<svg viewBox="0 0 24 24">${d}</svg>`,I={
 pill:P('<path d="M10.5 20.5a5 5 0 0 1-7-7l9-9a5 5 0 0 1 7 7z"/><path d="m8.5 8.5 7 7"/>'),sun:P('<circle cx="12" cy="12"
    r="4"/><path d="M12 2v2M12 20v2M4.9 4.9l1.4 1.4M17.7 17.7l1.4 1.4M2 12h2M20 12h2M4.9 19.1l1.4-1.4M17.7 6.3l1.4-1.4"/>'),
 noon:P('<circle cx="12" cy="10" r="3.5"/><path d="M12 2v2M3 10h2M19 10h2M5.6 3.6l1.4 1.4M17 5l1.4-1.4M3
    20h18"/>'),dusk:P('<path d="M17 18a5 5 0 0 0-10 0M12 9v3M4.2 10.2l1.4 1.4M18.4 11.6l1.4-1.4M1 18h22M7 22h10"/>'),
 moon:P('<path d="M21 13A9 9 0 0 1 11 3a7 7 0 0 0 10 10z"/>'),clock:P('<circle cx="12" cy="12" r="9"/><path d="M12 7v5l3
    2"/>'),shield:P('<path d="M12 3l8 3v6c0 5-3.5 8-8 9-4.5-1-8-4-8-9V6z"/><path d="m9 12 2 2 4-4"/>'),
 alert:P('<path d="M12 3 2 20h20z"/><path d="M12 10v4M12 17h.01"/>'),scan:P('<path d="M4 8V5a1 1 0 0 1 1-1h3M16 4h3a1 1 0 0 1
    1 1v3M20 16v3a1 1 0 0 1-1 1h-3M8 20H5a1 1 0 0 1-1-1v-3M4 12h16"/>'),
 file:P('<path d="M14 3H7a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h10a2 2 0 0 0 2-2V8z"/><path d="M14 3v5h5M9 13h6M9
```

```
    17h6"/>'),vol:P('<path d="M11 5 6 9H3v6h3l5 4zM16 9a4 4 0 0 1 0 6M19 6a8 8 0 0 1 0 12"/>'),
 users:P('<circle cx="9" cy="8" r="3.5"/><path d="M2 20a7 7 0 0 1 14 0M16 4.5a3.5 3.5 0 0 1 0 7M18 14a7 7 0 0 1 4
    6"/>'),check:P('<path d="m5 12 5 5 9-10"/>'),bolt:P('<path d="M13 2 4 14h7l-1 8 9-12h-7z"/>')};
let T,SM,S={lang:"en",view:"in",text:"",data:null,tb:null,ck:null};
const t=(g,k)=>(T[S.lang][g]||{})[k]||k,u=(k,d)=>t("ui",k).replace("{d}",d||"");
const post=(p,b)=>fetch(p,{method:"POST",headers:{"Content-
    Type":"application/json"},body:JSON.stringify(b)}).then(r=>r.json());
const ev=(type,detail,level)=>post("/api/event",{type,detail,level:level||"info"});
const keys=()=>S.data?S.data.meds.filter(m=>m.key).map(m=>m.key):[];
async function init(){T=await(await fetch("/api/i18n")).json();SM=await(await fetch("/api/samples")).json();render()}
function render(){
 $("#logo").innerHTML=I.pill;$("#tag").textContent=u("tag");
 $("#lang").innerHTML=Object.keys(LN).map(l=>`<button class="${l==S.lang?"on":""}"
    onclick="S.lang='${l}';S.tb=null;render()">${LN[l]}</button>`).join("");
 const tabs=[["in","t_in"],["plan","t_plan"],["tb","t_tb"],["ck","t_ck"],["cg","t_cg"]];
 $("#nav").innerHTML=tabs.map(([v,k],i)=>`<button class="${v==S.view?"on":""}"
    onclick="S.view='${v}';render()"><i>${i+1}</i>${u(k)}</button>`).join("")+`<button class="ghost ${S.view=="ev"?"on":""}"
    onclick="S.view='ev';render()">Evaluation</button>`;
 $("#main").innerHTML={in:vIn,plan:vPlan,tb:vTb,ck:vCk,cg:vCg,ev:vEv}[S.view]();
 if(S.view=="cg")care();if(S.view=="ev")runEval();document.documentElement.lang=S.lang}
const none=()=>`<div class="card empty"><div class="ic">${I.file}</div><h2>${u("empty")}</h2><div class="row" style="justify-
    content:center"><button class="btn primary" onclick="S.view='in';render()">${u("t_in")}</button></div></div>`;
function vIn(){return `<div class="two"><div class="card"><h2>Paste the medicine lines</h2><p class="mut" style="margin:4px 0
    0">From the discharge summary or prescription. Brand names, 1-0-1 notation and small spelling mistakes are fine.</p>
<div class="lbl">Try a sample</div><div class="chips">${Object.keys(SM).map(k=>`<button class="try"
    onclick="S.text=SM['${k}'];render()">${k}</button>`).join("")}</div>
<div class="lbl">Medicine lines</div><textarea id="txt" placeholder="Tab Metformin 500 mg 1-0-1 after
    food">${S.text}</textarea>
<div class="row"><button class="btn primary" onclick="go()">${I.bolt} ${u("go")}</button><label class="btn">${I.scan} Scan
    photo or PDF<input type="file" id="f" accept="image/*,application/pdf" hidden onchange="scanF(this)"></label></div></div>
<div><div class="card"><h2>How it works</h2>
<div class="step"><div class="ic">${I.file}</div><div><b>Read</b><span>Your medicine lines are understood and cleaned
    up.</span></div></div>
<div class="step"><div class="ic b">${I.shield}</div><div><b>Check</b><span>Matched against a curated, source-linked safety
    list.</span></div></div>
<div class="step"><div class="ic a">${I.sun}</div><div><b>Plan</b><span>A simple day plan you can hear in your
    language.</span></div></div></div>
<div class="card safe" style="margin-top:16px"><div>${I.check} AI never makes the medical claims</div><div>${I.check} Every
    warning links to the official label</div><div>${I.check} Unsure items are flagged for your
    pharmacist</div></div></div></div>`}
function scanF(i){const f=i.files[0];if(!f)return;const r=new FileReader();r.onload=async()=>{S.text="Reading the
    paper...";render();
 const x=await post("/api/scan",{data:r.result.split(",")[1],mime:f.type});S.text=x.error||x.text||"No medicines found. Try a
    clearer photo.";render()};r.readAsDataURL(f)}
async function go(){S.text=$("#txt").value;if(!S.text.trim())return;S.data=await
    post("/api/analyze",{text:S.text});S.tb=null;S.view="plan";render()}
function nm(m){return m.name+(m.known&&m.typed&&m.typed!=m.key?` (${m.typed})`:"")}
const SL={morning:["#f4ecdb",I.sun],afternoon:["#e1eaf1",I.noon],evening:["#efe3e5",I.dusk],night:["#e0e4ee",I.moon],asneeded
    :["#e3eae6",I.clock]},AV=[["#dce8e2","#4f7263"],["#e1eaf1","#476683"],["#f4ecdb","#8f6f30"],["#efe3e5","#8f5057"],["#e4e2
    ee","#5c5a82"]];
function vPlan(){
 if(!S.data)return none();const
    D=S.data,g={};D.meds.forEach(m=>(m.slots.length?m.slots:["asneeded"]).forEach(s=>(g[s]=g[s]||[]).push(m)));
 const stat=(ic,c,n,l)=>`<div class="card stat"><div class="ic ${c}">${ic}</div><div><div
    class="n">${n}</div><small>${l}</small></div></div>`;
 const cols=["morning","afternoon","evening","night","asneeded"].filter(s=>g[s]).map(s=>`<div class="card slot"><div
    class="hd" style="--t:${SL[s][0]}"><div class="ic">${SL[s][1]}</div><h3>${t("slot",s)}</h3><small
    class="mut">${g[s].length}</small></div><div class="bd">${g[s].map(m=>{const
    a=AV[(m.name.charCodeAt(0)+m.name.length)%5];return `<div class="med"><div class="mh"><div class="av"
    style="background:${a[0]};color:${a[1]}">${m.name[0]}</div><div><b>${m.name}</b><small>${m.dose||"?"}</small></div>
 <label class="take"><input type="checkbox" onchange="this.checked&&ev('dose','${nm(m).replace(/'/g,"")}
    (${s})')">${u("taken")}</label></div>
 <span class="chip">${t("food",m.food)}</span>${m.cat?`<span class="chip
    n">${t("cat",m.cat)}</span>`:""}${m.notes.map(n=>`<span class="chip n">${t("note",n)}</span>`).join("")}${m.verify?`<span
    class="chip v">${u("verify")}</span>`:""}</div>`}).join("")}</div></div>`).join("");
 const iss=D.issues.length?D.issues.map(i=>`<div class="iss ${i.sev}"><div
    class="ic">${i.sev=="low"?I.shield:I.alert}</div><div><b>${i.a==i.b?i.a:i.a+" + "+i.b}</b><span
    class="badge">${u("s_"+i.sev)}</span><br>${t("eff",i.eff)}<br><a
    href="https://dailymed.nlm.nih.gov/dailymed/search.cfm?labeltype=all&query=${i.a}" target="_blank"
    rel="noopener">DailyMed: ${i.a}</a></div></div>`).join(""):`<div class="iss"><div
    class="ic">${I.shield}</div><div>${u("none")}</div></div>`;
 return `<div class="top"><h2>${u("t_plan")}</h2><button class="btn" onclick="speak()">${I.vol} ${u("speak")}</button></div>
 <div class="stats">${stat(I.pill,"",D.meds.length,u("n_med"))}${stat(I.alert,"a",D.issues.length,u("n_alert"))}${stat(I.shie
    ld,"b",D.meds.filter(m=>m.verify).length,u("n_ver"))}</div>
 <div class="grid">${cols}</div><div class="card" style="margin-top:18px"><h2>${u("safety")}</h2>${iss}</div><p
    class="disc">${u("disc")}</p>`}
function speak(){speechSynthesis.cancel();const
    g={};S.data.meds.forEach(m=>(m.slots[0]?m.slots:["asneeded"]).forEach(s=>(g[s]=g[s]||[]).push(m)));
 const x=["morning","afternoon","evening","night","asneeded"].filter(s=>g[s]).map(s=>t("slot",s)+": "+g[s].map(m=>`${m.name}
    ${m.dose||""}, ${t("food",m.food)}`).join(". ")).join(". ");
 const m=new SpeechSynthesisUtterance(x);m.lang=VC[S.lang];speechSynthesis.speak(m)}
const sh=a=>[...a].sort(()=>Math.random()-.5);function opts(c,all){return sh([c,...sh(all.filter(x=>x!=c)).slice(0,3)])}
function mkTb(){const
    ms=S.data.meds.filter(m=>m.known&&m.slots[0]&&m.slots[0]!="asneeded").slice(0,3);if(!ms.length)return{qs:[],i:0,sc:0};
 return{i:0,sc:0,qs:ms.map((m,i)=>{const k=["q_time","q_why","q_food"][i];
  if(k=="q_time")return{q:u(k,m.name),o:opts(m.slots[0],["morning","afternoon","evening","night"]),a:m.slots[0],g:"slot"};
  if(k=="q_why")return{q:u(k,m.name),o:opts(m.cat,Object.keys(T.en.cat)),a:m.cat,g:"cat"};
  return{q:u(k,m.name),o:opts(m.food,["before","after","with","empty","any"]),a:m.food,g:"food"}})}}
```

```
function vTb(){if(!S.data)return none();if(!S.tb)S.tb=mkTb();const b=S.tb;if(!b.qs.length)return none();
 const seq=`<div class="seq">${b.qs.map((_,i)=>`<i class="${i<b.i?"d":i==b.i?"c":""}"></i>`).join("")}</div>`;
 if(b.i>=b.qs.length){const low=b.sc<b.qs.length-1;return `<div class="card q" style="text-
    align:center">${seq}<h2>${u("t_tb")}</h2><div class="ring" style="--
    p:${100*b.sc/b.qs.length}"><span>${b.sc}/${b.qs.length}</span></div>${low?`<div class="banner call" style="justify-
    content:center">${I.alert} ${u("alert")}</div>`:`<div class="banner ok" style="justify-content:center">${I.check}
    ${u("ok")}</div>`}<div class="row" style="justify-content:center"><button class="btn"
    onclick="S.tb=null;render()">${u("retry")}</button></div></div>`}
 const q=b.qs[b.i];return `<div class="card q">${seq}<h2>${q.q}</h2>${q.o.map((o,i)=>`<button class="opt"
    onclick="ans(this,'${o}')"><i>${"ABCD"[i]}</i>${t(q.g,o)}</button>`).join("")}</div>`}
function ans(el,o){const b=S.tb,q=b.qs[b.i];if(el.parentNode.dataset.d)return;el.parentNode.dataset.d=1;const
    ok=o==q.a;if(ok)b.sc++;
 el.classList.add(ok?"good":"bad");el.insertAdjacentHTML("beforeend",`<span style="margin-left:auto;font-
    size:14px">${u(ok?"ok":"no")}</span>`);
 setTimeout(()=>{b.i++;if(b.i>=b.qs.length)ev("teach-back",`Score
    ${b.sc}/${b.qs.length}`,b.sc<b.qs.length-1?"warn":"info");render()},900)}
function vCk(){const s=Object.keys(T.en.sym),c=S.ck;return `<div class="card"><h2>${u("tick")}</h2><div
    class="syms">${s.map(k=>`<label class="sym"><input type="checkbox" value="${k}">${t("sym",k)}</label>`).join("")}</div>
<div class="row"><button class="btn primary" onclick="chk()">${I.shield} ${u("check")}</button></div>${c?`<div class="banner
    ${c}">${c=="ok"?I.check:I.alert} ${t("lvl",c)}</div>`:""}</div><p class="disc">${u("disc")}</p>`}
async function chk(){const sy=[...document.querySelectorAll(".sym input:checked")].map(x=>x.value);S.ck=(await
    post("/api/checkin",{keys:keys(),symptoms:sy})).level;render()}
function vCg(){return `<div class="card"><div class="top"><h2>${u("live")}</h2><span
    class="chip">${I.users.replace("<svg",'<svg style="vertical-align:-2px"')} ${u("t_cg")}</span></div><div id="evs"
    style="margin-top:8px"></div></div>`}
let tm;async function care(){clearInterval(tm);const f=async()=>{if(S.view!="cg")return clearInterval(tm);const e=await(await
    fetch("/api/events")).json();
 $("#evs").innerHTML=e.length?e.map(x=>`<div class="ev ${x.level}"><span class="dot"></span><div><b>${x.type}</b>
    ${x.detail}<br><small class="mut">${x.t}</small></div></div>`).join(""):`<p class="mut">No updates yet. Mark a dose as
    taken, finish the teach-back or run a check-in and it appears here.</p>`};f();tm=setInterval(f,2500)}
function vEv(){return `<div class="card" id="evbox">Running the evaluation...</div>`}
async function runEval(){let r,note="";
 try{const x=await fetch("/api/eval");if(!x.ok)throw new Error("server error "+x.status);r=await x.json()}
 catch(e){try{r=await(await fetch("/eval_results.json")).json();note=` (live run failed: ${e.message}; showing saved
    results)`}catch(e2){const b=$("#evbox");if(b)b.textContent="Could not load results: "+e.message;return}}
 const b=$("#evbox");if(!b)return;const s=(n,v)=>`<div class="card"><div class="big">${v}%</div><div>${n}</div><div
    class="bar"><i style="width:${v}%"></i></div></div>`;
 b.outerHTML=`<div class="grid">${s("Medicine recall",r.drug_recall)}${s("Medicine precision",r.drug_precision)}${s("Dose
    accuracy",r.dose_accuracy)}${s("Schedule accuracy",r.schedule_accuracy)}${s("Interaction
    recall",r.interaction_recall)}</div>
 <div class="card" style="margin-top:16px"><p style="margin-top:0">${r.cases} synthetic notes, ${r.medicines} medicines.
    ${r.note}${note}</p>${r.failures.map(f=>`<div class="ev"><span class="dot bad"></span><span
    class="mut">${f}</span></div>`).join("")}</div>`}
init();
</script></body></html>
```

requirements.txt

```
fastapi>=0.110
uvicorn>=0.29
httpx>=0.27
```

README.md

```
# RxRosetta
Post-discharge medication companion: paste discharge-paper medicine lines, get a verified plan in English, Tamil or Hindi,
    with safety checks, a teach-back quiz, a red-flag check-in and a live caregiver feed.

## Run
    pip install -r requirements.txt
    uvicorn app:app --port 8000
Open http://localhost:8000 (open it in two tabs: use one as the patient, the other on the Caregiver tab).

## How it works
- engine.py: parser (Indian 1-0-1 notation, brand names, fuzzy spelling), interaction and duplicate checks, rule-based
    triage, evaluation harness.
- kb.py: curated knowledge base and translations. Every warning shown comes from here, with a DailyMed link to verify.
- Triage is deterministic rules, never LLM guesswork. Unknown or uncertain medicines are flagged "Verify with your
    pharmacist".
- Evaluation: `GET /api/eval` (also the Evaluation tab). Synthetic data only; no real patient information.

## Limits
Decision support only, not medical advice. The knowledge base covers 18 common drugs. Tamil and Hindi text should be reviewed
    by a native speaker before real use.
```